

Hoja de Ejercicios Complementarios: Bloque 1

Ficheros de Texto Plano, Gestión de Charsets, Buffers y API Moderna Java NIO.2

MÓDULO PROFESIONAL	CICLO Y UNIDAD	MODALIDAD / ENTORNO	DESTINATARIO / CARÁCTER
Acceso a Datos (0486)	DAM 2º Curso · UT1	Presencial / Distancia · Java 21	Alumnado · Consolidación Técnica

ALINEACIÓN CURRICULAR OFICIAL (REAL DECRETO 455/2010 · CURRÍCULO ASTURIAS)

RA1: Manejo de Ficheros

CE1.a (Clases de E/S y Flujos)

CE1.b (Ficheros de Texto Plano)

Esta batería de ejercicios afianza el tratamiento robusto de flujos de caracteres mediante `BufferedReader`, `BufferedWriter` y `PrintWriter`, el control estricto de tablas de codificación (`StandardCharsets`) para evitar *Mojibake*, y el análisis declarativo y de alto rendimiento mediante la API moderna **Java NIO.2** (`Path`, `Files.lines()` y Streams).

🚩 Instrucciones de Trabajo y Entorno de Desarrollo

Desarrolla las soluciones dentro de tu proyecto de Eclipse IDE (o entorno compatible con Java 21). Crea los archivos en la carpeta de trabajo `datos/` de tu proyecto. Asegúrate de gestionar siempre el cierre de flujos mediante bloques `try-with-resources` y no utilices rutas absolutas con letras de unidad (utiliza rutas relativas multiplataforma o `Path.of("datos", "fichero.txt")`).

Contexto Empresarial:

El departamento de Recursos Humanos recibe periódicamente un archivo CSV exportado desde un sistema ERP legado (`datos/empleados_raw.csv`). Dicho fichero fue grabado con codificación `ISO-8859-1` y contiene filas con campos incompletos, espacios superfluos en los nombres y salarios mal formateados.

Datos de Prueba (`datos/empleados_raw.csv`):

```
DNI;NOMBRE;DEPARTAMENTO;SALARIO;FECHA_ALTA
12345678A; García Pérez, María ;Administración;2450.50;2024-01-15
87654321B;Muñoz Cañete, José;Diseño & Marketing;invalid_salary;2023-11-20
23456789C;Álvarez Díaz, Lucía;I+D;3100.00
;López Vega, Ana;Ventas;1900.00;2024-03-01
34567890D;Núñez Rivas, Carlos;Logística y Almacén;2100.75;2022-06-10
```

Requerimientos Técnicos:

- Lectura con Charset específico:** Abre el fichero utilizando `Files.newBufferedReader(path, Charset.forName("ISO-8859-1"))`.
- Validación estricta de registros:**
 - La fila debe contener exactamente 5 campos separados por `;`.
 - El DNI no puede estar vacío ni contener solo espacios (`!dni.isBlank()`).
 - El salario debe parsearse a `double` y ser estrictamente positivo (`> 0`).
- Normalización de texto:** Elimina espacios en blanco sobrantes en los extremos del nombre mediante `trim()`.
- Salida de válidos en UTF-8:** Escribe los registros correctos en `datos/empleados_limpios.csv` con cabecera y codificación `StandardCharsets.UTF_8`.
- Auditoría de errores:** Vuelca las filas descartadas en `datos/empleados_errores.log` con el formato: `[LÍNEA X] Error: <Causa> -> Contenido: <Texto Original>`.

Trampa Típica: Mojibake por omisión de Codificación

Si omites el segundo argumento `Charset.forName("ISO-8859-1")`, Java 21 asumirá UTF-8 por defecto, provocando que los caracteres acentuados (como *Álvarez* o *Núñez*) se corrompan irremediablemente (generando caracteres anómalos `?`).

Salida Esperada en Consola:

```
[INFO] Procesando archivo de empleados en ISO-8859-1...
[AVISO] Línea 3 rechazada: Salario numérico no válido (invalid_salary)
[AVISO] Línea 4 rechazada: Número de columnas insuficiente (4 campos)
[AVISO] Línea 5 rechazada: DNI vacío o en blanco
[ÉXITO] Proceso completado: 2 registros normalizados en UTF-8, 3 rechazados.
```

Ejercicio 2: Generador de Informes de Ventas con `PrintWriter`

(`GeneradorInforme.java`)

NIVEL
INTERMEDIO

Contexto Empresarial:

La dirección comercial necesita un generador automático de informes diarios de liquidación de sucursales a partir de un fichero de transacciones sin formato. El documento resultante (`datos/informe_ventas.txt`) debe presentar un diseño tabulado riguroso y columnas monetarias alineadas al milímetro.

Datos de Prueba (`datos/ventas_diarias.txt`):

```
T01|Avilés Centro|14520.80|142
T02|Gijón Puerto|23100.50|215
T03|Oviedo Catedral|18940.25|180
T04|Llanes Costa|8450.00|94
```

Requerimientos Técnicos:

- Composición de Streams:** Utiliza `PrintWriter` envolviendo a `BufferedWriter` y `FileWriter` con `StandardCharsets.UTF_8`.
- Máscaras de alineación `printf`:**
 - ID de tienda (6 caracteres) y Ciudad (18 caracteres) alineados a la izquierda: `%-6s` , `%-18s` .
 - Importe Neto (12 caracteres) con 2 decimales y símbolo monetario: `%12.2f €` .
 - Unidades vendidas alineadas a la derecha: `%8d` .
 - Ticket Medio calculado (`Importe / Unidades`) con 2 decimales: `%10.2f €` .
- Estadísticas de Cierre:** Al pie del informe, imprime una línea de separación continua (`=`) y muestra: Total de facturación de la cadena, Media de ventas por tienda y Nombre de la tienda líder.
- Copia de Seguridad en NIO.2:** Una vez cerrado el informe, genera un backup atómico con `Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING)` hacia `datos/backups/informe_ventas_backup.txt` .

Aspecto Esperado del Fichero Generado (`informe_ventas.txt`):

```
=====
                        INFORME DIARIO DE VENTAS - CADENA ASTUR RETAIL
=====
ID      CIUDAD                IMPORTE NETO    UNIDADES    TICKET MEDIO
-----
T01     Avilés Centro          14520,80 €      142         102,26 €
T02     Gijón Puerto              23100,50 €      215         107,44 €
T03     Oviedo Catedral          18940,25 €      180         105,22 €
T04     Llanes Costa              8450,00 €       94          89,89 €
=====
TOTAL FACTURACIÓN:           65011,55 €
PROMEDIO POR TIENDA:         16252,89 €
TIENDA LÍDER:                 Gijón Puerto (23100,50 €)
=====
```

Ejercicio 3: Analizador Forense de Tráfico con Java NIO.2 y Streams

(LogAnalyzerNIO.java)

NIVEL
AVANZADO

Contexto Empresarial:

Un servidor web de producción genera diariamente un archivo de acceso (`datos/access.log`) con decenas de miles de peticiones HTTP. La dirección de ciberseguridad requiere una utilidad de consola de alto rendimiento capaz de analizar el tráfico en *streaming* sin cargar el fichero entero en la memoria RAM (evitando colapsos por `OutOfMemoryError`).

Formato de Entrada (`datos/access.log`):

```
192.168.1.105 - - [17/Sep/2026:10:14:22] "GET /api/productos HTTP/1.1" 200 4520
10.0.0.15 - - [17/Sep/2026:10:14:23] "POST /login HTTP/1.1" 401 512
172.16.0.4 - - [17/Sep/2026:10:14:25] "GET /admin/db.php HTTP/1.1" 404 1204
192.168.1.105 - - [17/Sep/2026:10:14:30] "GET /imagenes/logo.png HTTP/1.1" 200 18450
10.0.0.88 - - [17/Sep/2026:10:14:35] "POST /checkout HTTP/1.1" 500 850
192.168.1.200 - - [17/Sep/2026:10:14:36] "GET /contacto HTTP/1.1" 200 1340
10.0.0.15 - - [17/Sep/2026:10:14:40] "POST /login HTTP/1.1" 401 512
10.0.0.15 - - [17/Sep/2026:10:14:42] "POST /login HTTP/1.1" 200 2048
```

Consultas Requeridas mediante API de Streams:

1. **Lectura bajo demanda:** Utiliza `Files.lines(Path, StandardCharsets.UTF_8)` dentro de un bloque `try-with-resources` . *Queda terminantemente prohibido acumular todas las líneas en una colección intermedia como `List<String>` .*
2. **Métricas Globales:** Cuenta el número total de accesos registrados mediante `stream.count()` .
3. **Detección de Incidentes (HTTP 4xx y 5xx):** Filtra todas las peticiones con código de estado ≥ 400 y vuelca sus líneas al archivo de seguridad `datos/peticiones_sospechosas.txt` mediante `Files.write()` .
4. **Volumen de Datos Transferidos:** Calcula mediante `mapToLong()` y `sum()` la cantidad total de bytes transmitidos correspondientes exclusivamente a peticiones exitosas (código `200`).
5. **Top IPs más activas:** Emplea `Collectors.groupingBy(..., Collectors.counting())` para identificar y listar por consola las 3 direcciones IP con mayor volumen de solicitudes.

Regla Crítica de NIO.2: Cierre Obligatorio del Stream

Los Streams devueltos por `Files.lines()` encapsulan un puntero al sistema de archivos subyacente. Si no se cierran dentro de un `try-with-resources` , se producirá una fuga de descriptores de archivo en el sistema operativo (*file descriptor leak*).

Salida Esperada en Consola:

```
=== AUDITORÍA FORENSE DE TRÁFICO WEB (NIO.2 STREAMS) ===  
Total peticiones analizadas: 8  
Peticiones sospechosas detectadas (HTTP 4xx/5xx): 4  
-> Exportadas a 'datos/peticiones_sospechosas.txt'  
Bytes transferidos en respuestas 200 OK: 26.358 bytes (25,74 KB)  
  
Top 3 Direcciones IP más activas:  
1. 10.0.0.15      -> 3 peticiones  
2. 192.168.1.105  -> 2 peticiones  
3. 192.168.1.200  -> 1 peticiones  
=====
```